МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП'ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

ЗВІТ

до лабораторної роботи №1

з дисципліни "Програмування систем на кристалі"

Студента 4 курсу
Спеціальності F7 –
"Комп'ютерна інженерія"
Кузана Дениса Вікторовича

м. Ужгород – 2026 р.

Лабораторна робота №1

Тема: Налаштування системи тактування (Clocks) та обробники переривань (ISR) у **PSoC 4**

Мета: Ознайомитися з тактуванням та обробникою переривань (ISR) для мікроконтролерів PSoC 4, використовуючи засоби інтегрованого середовища PSoC® Creator™, а також підключати й використовувати логічний аналізатор Saleae Logic для апаратної верифікації розробленого проекту.

Хід виконання завдання

Завдання 1 Розрахунки тактування, основних параметрів і похибок.

Перед тим як складати схему та виконувати подальші модифікації проекту, треба визначитись з тактуванням, розрахувати дільник джерела, фактичну частоту, відносну похибку частоти шини, дільник для локального таймера, його фактичну частоту й період перемикання сигналу основного циклу. Вхідні параметри надано в таблиці 1.

Таблиця 1 – Варіант завдання

Варіант	IMO, МГц	Бажаний BUS CLK	Тік Timer (Clock_1)	Період переривання Timer_1	CyDelay в main()
1	24	8 МГц	1 МГц (1 мкс)	1 мс	3 мс

Розрахунок дільника джерела BUS_Clk:

Теоретично, для того щоб отримати 8 МГц, дільник має бути 3, так як IMO = 24.

$$divider_bus = \frac{IMO}{BUS_CLK_{desired}} = \frac{24\text{МГц}}{8\text{МГц}} = 3$$

Фактична частота BUS_Clk (Nominal/Actual):

Фактичну частоту джерела можна знайти, якщо поділити частоту базового генератору на вирахований раніше дільник.

$$BUS_CLK = \frac{IMO}{divider_bus} = \frac{24\ \text{МГц}}{3} = 8\ \text{МГц}$$

Відносна похибка частоти шини:

Для обрахунку відносної похибки частоти шини використовується наступний вираз:

$$\delta = \frac{\left|BUS_CLK_{actual} - BUS_CLK_{desired}\right|}{BUS_CLK_{desired}} * 100\% = \frac{|8-8|}{8} * 100\% = 0\%$$

Розрахунок дільника для локального клоку таймера (Clock_1):

Для формування частоти 1МГц джерело BUS_CLK (8 МГц) ділиться середовищем автоматично:

$$divider_Clock_1 = \frac{HFClk}{Clock_1_{desired}} = \frac{8\ \text{МГц}}{1\ \text{МГц}} = 8$$

Фактична частота Clock_1: 1 МГц (похибка 0%).

Розрахунок значення регістру періоду таймера (Timer_1 Period):

Період таймера розраховується за наступною формулою:

$$Period = \frac{T_{ISR}}{T_{tick}} = \frac{1\ \text{мс}}{1\ \text{мкс}} = \frac{1000\ \text{мкс}}{1\ \text{мкс}} = 1000\ \text{тіків}$$

Розрахунок періоду перемикання сигналу основного циклу (Pin_Main):

Так як в основному циклі виконується послідовність: Pin_Main_Write(1), CyDelay(3ms), Pin_Main_Write(0), CyDelay(3ms), то загальний розрахунковий період прямокутного сигналу на Pin_Main: $T_{Main} = 3\text{мс} + 3\text{мс} = 6\text{мс}$

Завдання 2 Побудувати схему й налаштувати піни

Для того щоб виконати завдання, потрібно побудувати схему з компонент, наведених на таблиці 2.

Таблиця 2 – Компоненти схеми

Компонент	**Назва**	**Налаштування**
Clock	Clock_1	Джерело - BUS CLK
TCPWM	Timer_1	Вхідний клок - Clock_1
Interrupt (isr)	isr_Timer	Вхід підключити до виходу переривання Timer_1
Digital Output Pin	Pin_ISR	Порт P0_2 (виведений на RGB-світлодіод на платі), Strong Drive
Digital Output Pin	Pin_Main	Порт P0_3 (виведений на RGB-світлодіод на платі), Strong Drive

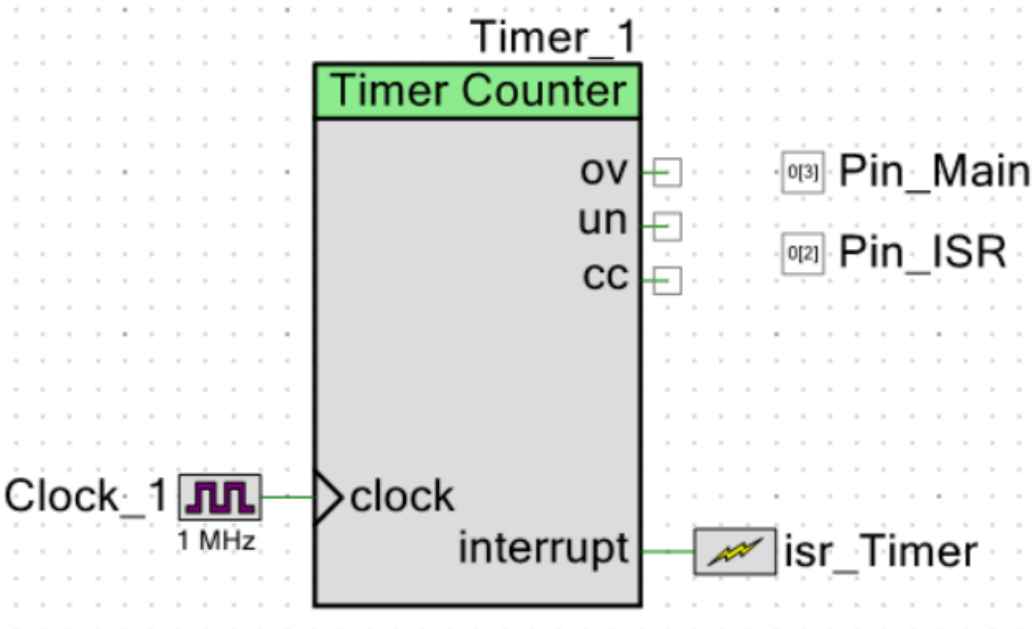


Рисунок 1 – Побудована схема

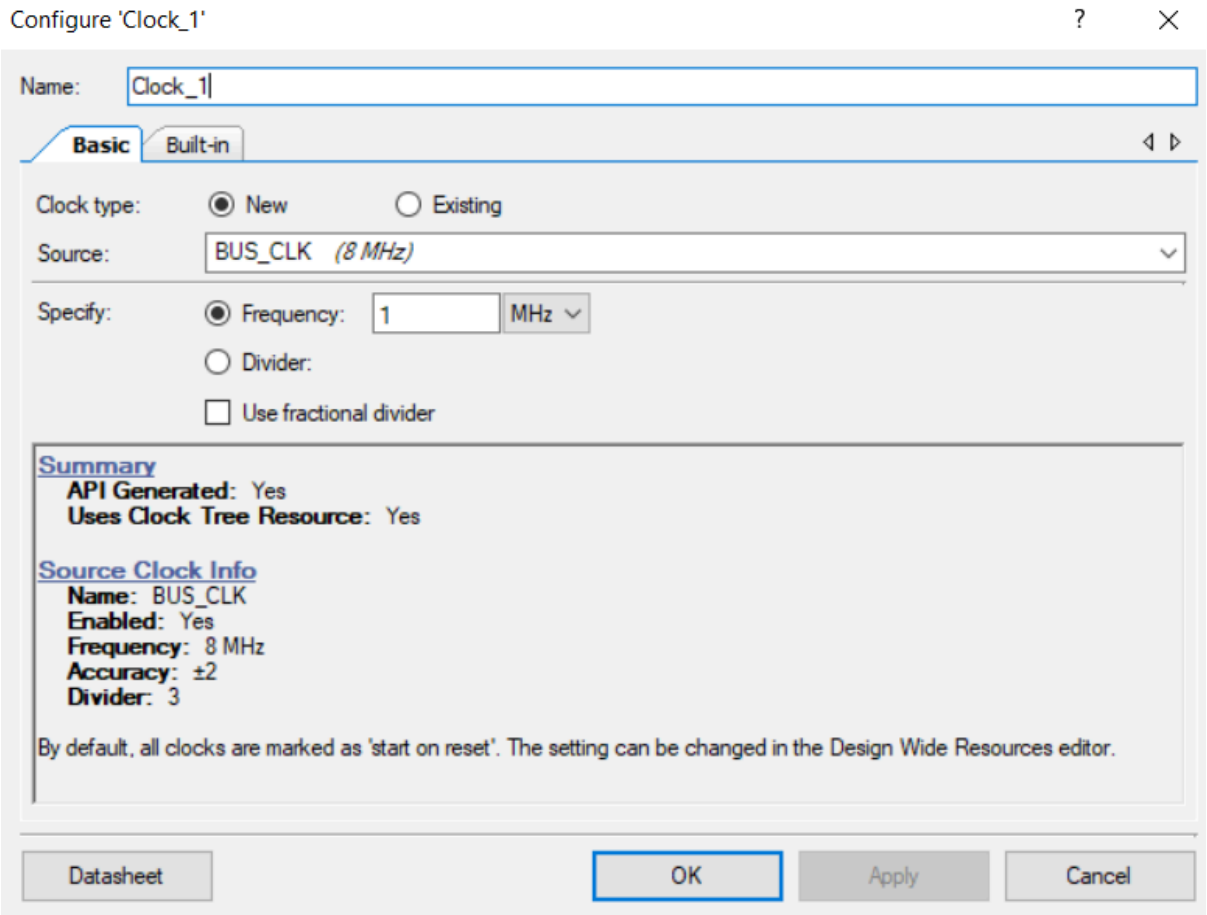


Рисунок 2 – Налаштування Clock

Також потрібно налаштувати піни. Порти для пінів зазначені в таблиці 2, а налаштування пінів зображено на рисунку 3.

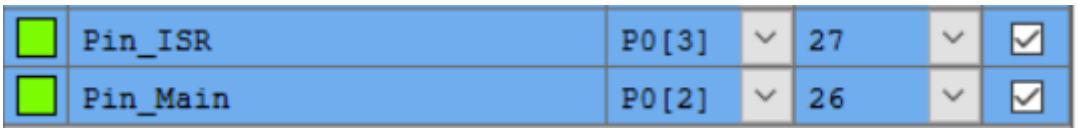


Рисунок 3 – Налаштовані піни

Лістинг коду з main.c:

```
#include "project.h"

/* Варіант 1: Clock_1 = 1 МГц (тік 1 мкс), період переривання
1 мс -> 1000 тіків */
```

```
#define TIMER_PERIOD_TICKS (1000u)

volatile uint32_t isr_timer_count = 0u;

CY_ISR(Timer_ISR_Handler)
{

Pin_ISR_Write((uint8)(!Pin_ISR_Read()));

isr_timer_count++;

Timer_1_ClearInterrupt(Timer_1_INTR_MASK_TC);
}

int main(void)
{
CyGlobalIntEnable;

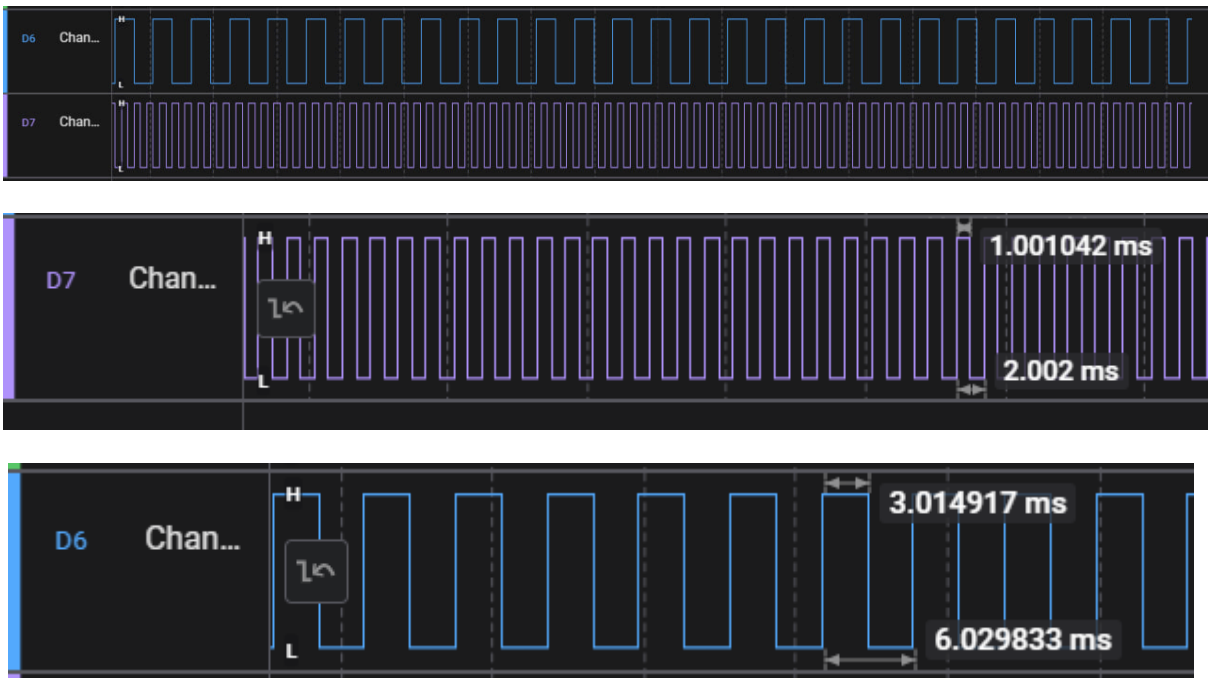
Clock_1_Start();
Timer_1_Start();
Timer_1_WritePeriod(TIMER_PERIOD_TICKS);
Timer_1_SetInterruptMode(Timer_1_INTR_MASK_TC);

/* Реєстрація та запуск обробників переривань */
isr_Timer_StartEx(Timer_ISR_Handler);

   for(;;)
   {
   Pin_Main_Write(1u);
   CyDelay(3);
   Pin_Main_Write(0u);
   CyDelay(3);
   }
}
```

Завдання 3 Виконати вимірювання за допомогою Saleae Logic 2.0

Використовуючи додаток Logic 2.0, а також логічний аналізатор, було виконано вимірювання значень реального періоду і тривалості ISR. Результати вимірювань наведено на рисунках 4-6.



Рисунки 4-6 – Результати вимірювання за допомогою Logic 2.0

Всі результати вимірювань і розрахунків наведено в таблиці 3.

Завдання 4 Модифікувати переривання так, щоб від фізичної кнопки вмикався/вимикався світлодіод

Для того щоб виконати завдання, було додано кнопку і переривання для неї, а також модифіковано код, додавши до нього переривання фізичної кнопки.

```
CY_ISR(Button_Main_Handler)
{
uint8 current_state = Pin_Main_Read();
Pin_Main_Write((uint8)(!current_state));

Button_ClearInterrupt();
}
```

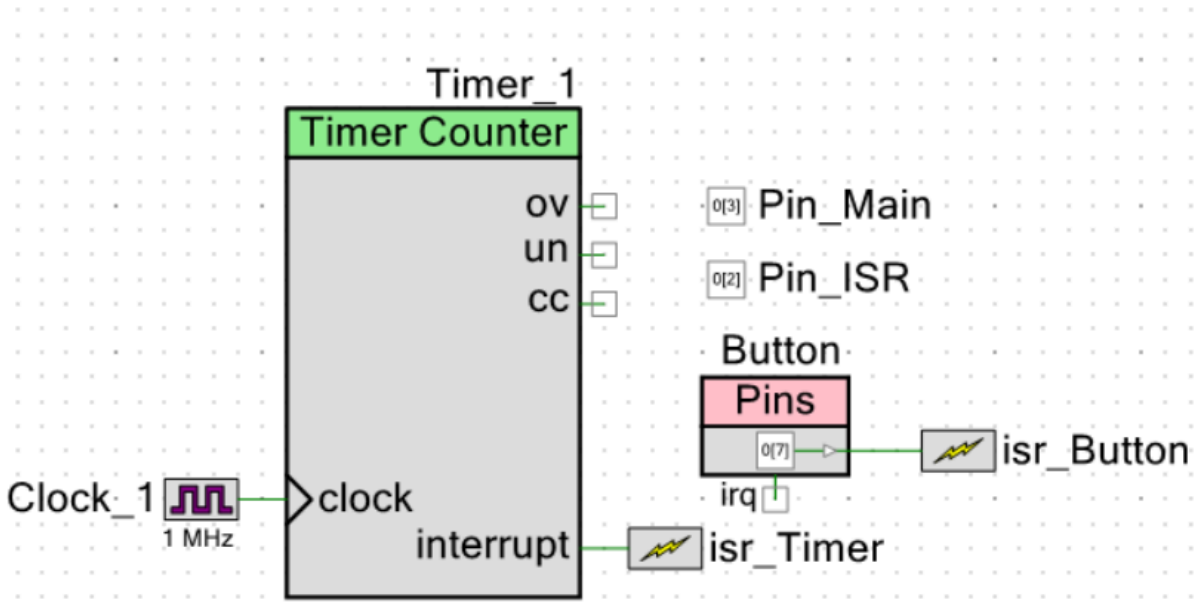


Рисунок 7 – Модифікована схема

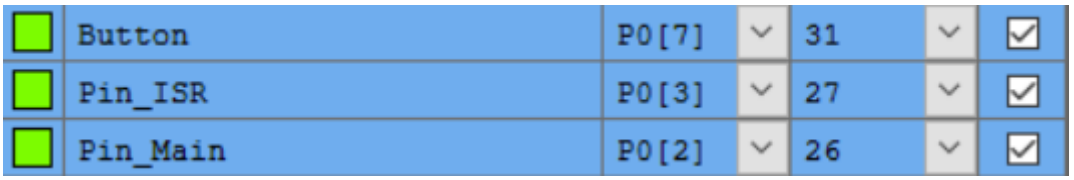


Рисунок 8 – Модифіковані порти

Лістинг модифікованого коду:

```
#include "project.h"
#define TIMER_PERIOD_TICKS (000u)

volatile uint32_t isr_timer_count = 0u;

CY_ISR(Timer_ISR_Handler)
{
    Pin_ISR_Write((uint8)(!Pin_ISR_Read()));

    isr_timer_count++;


    Timer_1_ClearInterrupt(Timer_1_INTR_MASK_TC);
}


CY_ISR(Button_Main_Handler)
{
```

```
    uint8 current_state = Pin_Main_Read();
    Pin_Main_Write((uint8)(!current_state));

    Button_ClearInterrupt();
}

int main(void)
{
    CyGlobalIntEnable;

    Clock_1_Start();
    Timer_1_Start();
    Timer_1_WritePeriod(TIMER_PERIOD_TICKS);
    Timer_1_SetInterruptMode(Timer_1_INTR_MASK_TC);


    isr_Timer_StartEx(Timer_ISR_Handler);
    isr_Button_StartEx(Button_Main_Handler);


}
```

Таблиця 3 – Результати розрахунків і вимірювань

Параметр / Сигнал	**Розрахункове**	**IDE Actual / Nominal**	**Виміряне (Saleae Logic 2)**
IMO	24 МГц	24 МГц	–
BUS_CLK (SysClk)	8 МГц	8 МГц (дільник = 3)	–
Clock_1	1 МГц	1 МГц (дільник = 8)	–
Регістр Period (Timer_1)	1000 тіків	1000 тіків	–

Період сигналу ISR (Pin_ISR)	1.000 мс	1.000 мс	1.001 мс
Період сигналу main (Pin_Main)	6.000 мс	6.000 мс	6.030 мс

Висновки: під час виконання даної лабораторної роботи було ознайомлено з тактуванням та обробником переривань (ISR) для мікроконтролерів PSoC 4, використовуючи засоби інтегрованого середовища PSoC® Creator™, а також підключено й використано логічний аналізатор Saleae Logic для перевірки.